

# Signals Lab 2 - Impulse Response, Step Response, and RC Circuits

Colden Briggs - Eben Radocchia - Liam Donlan

Instructor: Dr. Ossareh

University of Vermont

October 5, 2026

# Contents

## Introduction

Our objectives for this lab were to examine how a short input pulse can be used to approximate the impulse response for a system  $h(t)$ , determine how short a pulse length  $T$  can be used for this approximation, and test whether, as  $T \rightarrow 0$  the response of a system depends on the shape of the pulse. We aimed to accomplish these objectives through hand analysis of the pulse and step responses of the two-stage RC cascade shown in Figure ??, numerical modeling of the two-stage RC circuit with python, and experimental construction and measurement of the two-stage RC circuit on a breadboard.

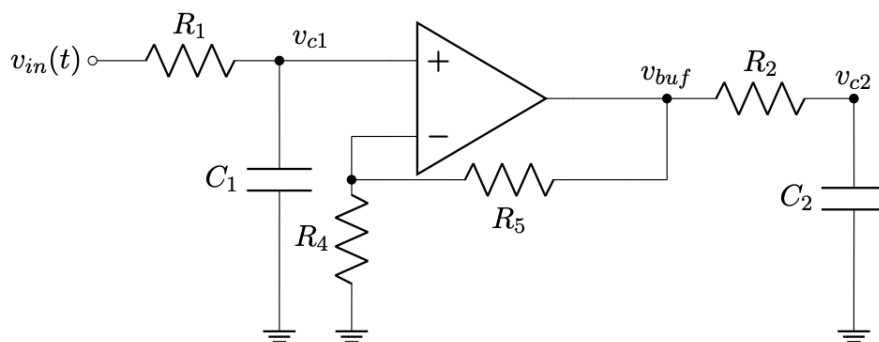


Figure 1: Two-stage RC Cascade with an input RC circuit, non-inverting amplifier circuit, and output RC circuit

## Task 1: Prelab Derivations

### 1. Time constants and gain

$$\begin{aligned}\tau_1 &= R_1 C_1 = (10 \text{ k}\Omega)(1 \text{ }\mu\text{F}) = 1.0 \times 10^{-2} \text{ s} \\ \tau_2 &= R_2 C_2 = (4.7 \text{ k}\Omega)(1 \text{ }\mu\text{F}) = 4.7 \times 10^{-3} \text{ s} \\ G &= 1 + \frac{R_5}{R_4} = 1 + \frac{4.7 \text{ k}\Omega}{1 \text{ k}\Omega} = 5.7\end{aligned}$$

### 2. Second-order ODE for $v_{c2}(t)$

The two stages are described by

$$\begin{aligned}\dot{v}_{c1}(t) &= \frac{1}{\tau_1}(v_{in}(t) - v_{c1}(t)), \\ \dot{v}_{c2}(t) &= \frac{1}{\tau_2}(G v_{c1}(t) - v_{c2}(t)).\end{aligned}$$

Differentiating the second equation and substituting  $\dot{v}_{c1}$  from the first:

$$\begin{aligned}\ddot{v}_{c2}(t) &= \frac{G}{\tau_2}\dot{v}_{c1}(t) - \frac{1}{\tau_2}\dot{v}_{c2}(t) \\ &= \frac{G}{\tau_1\tau_2}(v_{in}(t) - v_{c1}(t)) - \frac{1}{\tau_2}\dot{v}_{c2}(t).\end{aligned}$$

Solving the second stage equation for  $v_{c1}$  gives

$$v_{c1}(t) = \frac{\tau_2 \dot{v}_{c2}(t) + v_{c2}(t)}{G},$$

and substituting this in:

$$\begin{aligned}\ddot{v}_{c2}(t) &= \frac{G}{\tau_1\tau_2}v_{in}(t) - \frac{1}{\tau_1\tau_2}(\tau_2\dot{v}_{c2}(t) + v_{c2}(t)) - \frac{1}{\tau_2}\dot{v}_{c2}(t) \\ &= \frac{G}{\tau_1\tau_2}v_{in}(t) - \frac{1}{\tau_1}\dot{v}_{c2}(t) - \frac{1}{\tau_1\tau_2}v_{c2}(t) - \frac{1}{\tau_2}\dot{v}_{c2}(t).\end{aligned}$$

Collecting terms:

$$\boxed{\ddot{v}_{c2}(t) + \left(\frac{1}{\tau_1} + \frac{1}{\tau_2}\right)\dot{v}_{c2}(t) + \frac{1}{\tau_1\tau_2}v_{c2}(t) = \frac{G}{\tau_1\tau_2}v_{in}(t)}$$

### 3. Impulse response $h(t)$

For an impulse input, the first stage output is the single-stage RC impulse response (Section 1.1 with  $\tau = \tau_1$ ):

$$v_{c1}(t) = \frac{1}{\tau_1}e^{-t/\tau_1}u(t).$$

Plugging this into the second stage ODE, for  $t \geq 0$ :

$$\dot{v}_{c2}(t) + \frac{1}{\tau_2}v_{c2}(t) = \frac{G}{\tau_1\tau_2}e^{-t/\tau_1}.$$

Using the integrating factor  $\mu = e^{t/\tau_2}$ :

$$v_{c2}(t) = e^{-t/\tau_2} \left( \int \frac{G}{\tau_1 \tau_2} e^{t/\tau_2 - t/\tau_1} dt + C \right).$$

The exponent simplifies to

$$\frac{t}{\tau_2} - \frac{t}{\tau_1} = \frac{t(\tau_1 - \tau_2)}{\tau_1 \tau_2},$$

so the integral is

$$\int \frac{G}{\tau_1 \tau_2} e^{\frac{t(\tau_1 - \tau_2)}{\tau_1 \tau_2}} dt = \frac{G}{\tau_1 - \tau_2} e^{\frac{t(\tau_1 - \tau_2)}{\tau_1 \tau_2}},$$

which gives

$$v_{c2}(t) = \frac{G}{\tau_1 - \tau_2} e^{-t/\tau_1} + C e^{-t/\tau_2}.$$

The initial condition  $v_{c2}(0) = 0$  gives  $C = -\frac{G}{\tau_1 - \tau_2}$ . Therefore

$$h(t) = \frac{G}{\tau_1 - \tau_2} (e^{-t/\tau_1} - e^{-t/\tau_2}) u(t)$$

Figure ?? shows  $h(t)$  for the given component values.

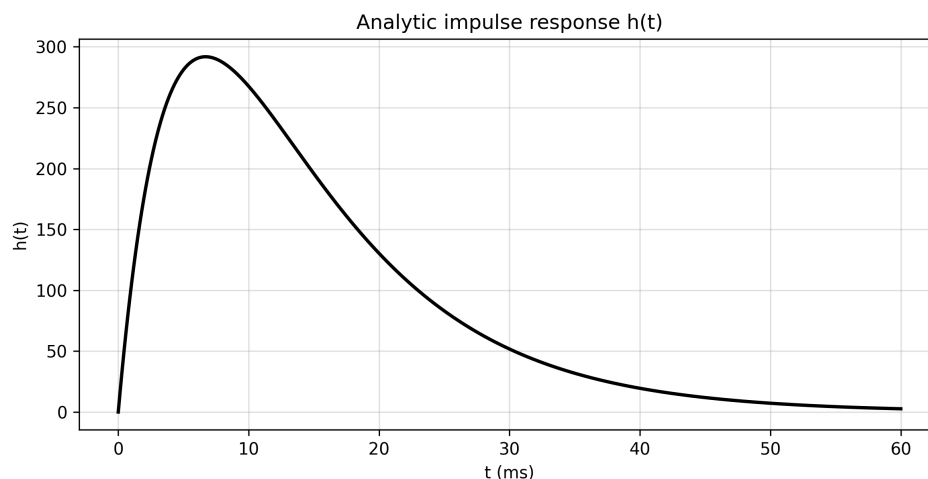


Figure 2: Analytic impulse response  $h(t)$ .

#### 4. Step response $s(t)$

The step response,  $s(t)$ , is the response of a system to the unit step input. When given the impulse response  $h(t)$ , the step response can be found by integrating  $h(t)$  from 0 to  $t$ , as shown below.

$$\begin{aligned} s(t) &= \int_0^t h(\sigma) d\sigma = \frac{G}{\tau_1 - \tau_2} \int_0^t (e^{-\sigma/\tau_1} - e^{-\sigma/\tau_2}) d\sigma \\ &= \frac{G}{\tau_1 - \tau_2} \left[ -\tau_1 e^{-\sigma/\tau_1} + \tau_2 e^{-\sigma/\tau_2} \right]_{\sigma=0}^{\sigma=t} \\ &= \frac{G}{\tau_1 - \tau_2} \left[ -\tau_1 e^{-t/\tau_1} + \tau_2 e^{-t/\tau_2} + \tau_1 - \tau_2 \right] \end{aligned}$$

$$s(t) = \frac{G}{\tau_1 - \tau_2} \left[ \tau_1 (1 - e^{-t/\tau_1}) + \tau_2 (e^{-t/\tau_2} - 1) \right]$$

## 5. DC gain

The final value of  $s(t)$ , the DC gain, can be calculated by taking the limit of  $s(t)$  as  $t \rightarrow \infty$ . For this system, as  $t \rightarrow \infty$  both exponentials go to zero:

$$\lim_{t \rightarrow \infty} s(t) = \frac{G}{\tau_1 - \tau_2} [\tau_1 - \tau_2] = \boxed{G = 1 + \frac{R_5}{R_4}}.$$

## 6. Pulse response by superposition

A rectangular pulse of height  $A$  and width  $T$  is the difference of two shifted steps:

$$v_{in}(t) = A u(t) - A u(t - T) = \begin{cases} A, & 0 \leq t < T \\ 0, & \text{otherwise.} \end{cases}$$

The circuit is linear (its ODEs are linear) and time-invariant (the coefficients are constant), so scaling an input scales the output and delaying an input delays the output by the same amount. The response to a unit step is  $s(t)$ , so

$$A u(t) - A u(t - T) \xrightarrow{\text{LTI system}} A s(t) - A s(t - T),$$

and therefore

$$\boxed{v_{out}(t) = A[s(t) - s(t - T)]}$$

## 7. $v_{out}(t)$ using $s(t)$

For  $0 \leq t < T$ ,  $s(t - T) = 0$ , so  $v_{out}(t) = A s(t)$ . For  $t \geq T$ :

$$\begin{aligned} v_{out}(t) &= A[s(t) - s(t - T)] \\ &= \frac{AG}{\tau_1 - \tau_2} \left( [\tau_1(1 - e^{-t/\tau_1}) + \tau_2(e^{-t/\tau_2} - 1)] \right. \\ &\quad \left. - [\tau_1(1 - e^{-(t-T)/\tau_1}) + \tau_2(e^{-(t-T)/\tau_2} - 1)] \right) \end{aligned}$$

$$\boxed{v_{out}(t) = \frac{AG}{\tau_1 - \tau_2} \left[ \tau_1 (e^{-(t-T)/\tau_1} - e^{-t/\tau_1}) + \tau_2 (e^{-t/\tau_2} - e^{-(t-T)/\tau_2}) \right]}$$

## 8. Maximum $T$ for 1% error

From Section 1.2, a 1% error requires  $T \lesssim 0.02 \tau$ , where  $\tau$  is the smallest time constant of the system. Here the smallest time constant is  $\tau_2 = 4.7 \times 10^{-3} \text{ s}$ , so

$$T \lesssim 0.02 (4.7 \times 10^{-3} \text{ s}) = 9.4 \times 10^{-5} \text{ s} = \boxed{94 \text{ } \mu\text{s}}.$$

## Task 2.2: Rectangular Pulses

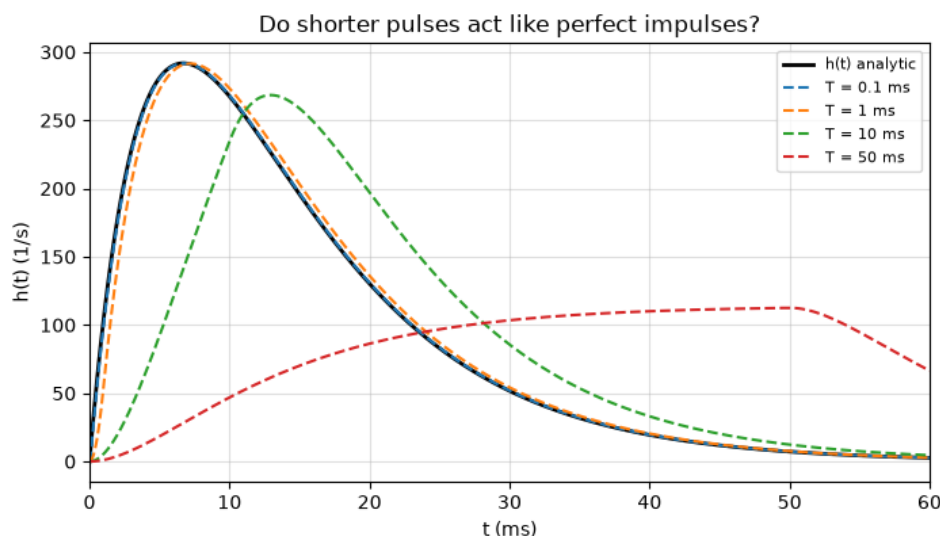


Figure 3: Four pulse widths compared to the analytic  $h(t)$

### Which $T$ values collapse onto $h(t)$

Figure ?? shows the output divided by the pulse area ( $A \cdot T$ ) for four different pulse widths, compared with the analytic  $h(t)$ . The shortest pulse,  $T = 0.1$  ms collapsed almost perfectly onto  $h(t)$ , while the second shortest pulse,  $T = 1$  ms, stays quite close to  $h(t)$  and has the same overall shape, making it also work as a decent approximation of the impulse response. The 10 ms and 50 ms pulses do not collapse onto  $h(t)$  at all. The 10 ms curve peaks lower and later than  $h(t)$ , and the 50 ms curve just slowly rises to a plateau around  $110\text{s}^{-1}$  before falling off, which looks a lot more like a step response than an impulse response.

This lines up with the rule  $T \lesssim 0.02 \tau_{\min}$ . The smallest time constant is  $\tau_2 = 4.7$  ms, so the pulse needs to be shorter than  $0.02(4.7 \text{ ms}) \approx 94 \mu\text{s}$  to be within 1% of  $h(t)$ . The 0.1 ms pulse is right at that limit, so it matches best. The 1 ms pulse is about 10 times over the limit, so it is close but a little off (roughly 5 to 10% error). The 10 ms and 50 ms pulses are far over the limit, so they are nowhere near  $h(t)$ . Basically, the pulse has to be much shorter than the circuit's time constants. If it is not, the circuit has time to respond while the pulse is still on, and the output is no longer just the impulse response.

```

1 def f_rhs(t, x, T):
2     v1, v2 = x
3     _vin = A if 0.0 <= t < T else 0.0
4     return [(_vin - v1)/tau1, (G*v1 - v2)/tau2]
5 def simulate(T):
6     t_end = T + 8*tau_max
7     t_eval = np.linspace(0.0, t_end, 4000)
8     max_step = min(T, tau_min)/20.0
9     sol = solve_ivp(f_rhs, (0.0, t_end), [0.0, 0.0], t_eval=t_eval,
10                    rtol=1e-8, atol=1e-8, max_step=max_step, args=(T,))
11     vin = np.zeros(len(t_eval))
12     vin[(t_eval >= 0) & (t_eval <= T)] = A

```

```

13     return t_eval, vin, sol.y[1]
14
15 fig1, ax = plt.subplots(2, 1, figsize=(8, 6))
16 fig2, axn = plt.subplots(figsize=(8, 4.2))
17
18 t_ref = np.linspace(0.0, 60e-3, 4000)
19
20 axn.plot(t_ref*1e3, h(t_ref), "k", lw=2, label="h(t) analytic")
21
22 for T in [100e-6, 1e-3, 10e-3, 50e-3]:
23     t, vin, v2 = simulate(T)
24     label = f"T = {T*1e3:g} ms"
25     ax[0].plot(t*1e3, vin, label=label)
26     ax[1].plot(t*1e3, v2, label=label)
27     axn.plot(t*1e3, v2/(A*T), "--", label=label)

```

Listing 1: Modified code for comparison of rectangular inputs.

## Effect of amplitude ( $A = 5$ V vs. $A = 1$ V, $T = 100$ ms)

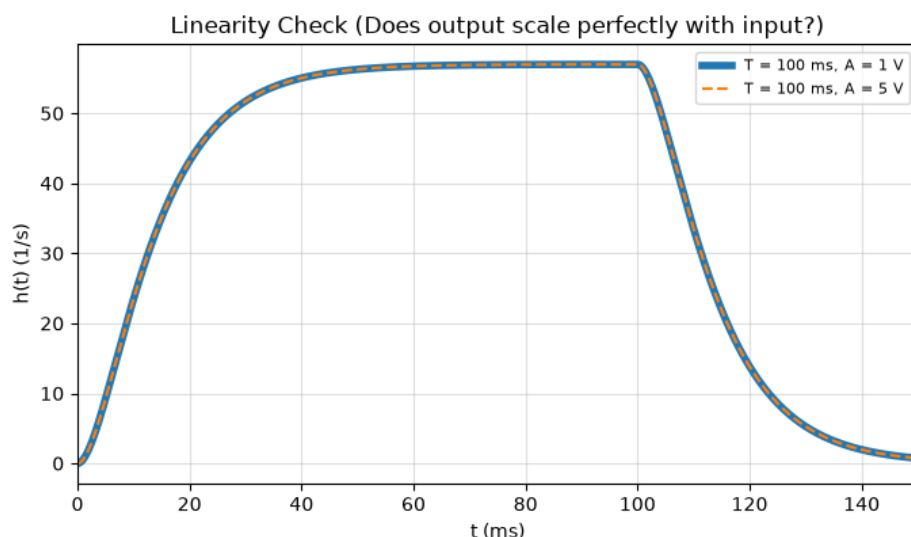


Figure 4: Area-normalized response for  $A = 5$  V and  $A = 1$  V at  $T = 100$  ms.

In Figure ??, the  $A = 1$  V and  $A = 5$  V curves land right on top of each other so the area-normalized curves are identical (the only difference is tiny numerical error from the solver). This shows the homogeneous property of the linear system. When the input gets 5 times bigger, the output also gets exactly 5 times bigger, so dividing by  $A \cdot T$  cancels the amplitude out and same curve both times. The flat part of the curves sits at about  $57\text{s}^{-1}$ , which is  $G/T = 5.7/0.1 = 57\text{s}^{-1}$ . This makes sense because the output settles at the DC gain  $G$  times the input before the pulse ends. Neither curve matches  $h(t)$  because 100 ms is way too long to act like an impulse, but that is fine since this test is only checking an aspect of linearity.

```

1 def f_rhs(t, x, T, A): # needed for solve_ivp
2     v1, v2 = x
3     if (0.0 <= t < T):

```

```
4     _vin = A
5     else:
6         _vin = 0.0
7     return [(_vin - v1)/tau1, (G*v1 - v2)/tau2]
8 def simulate(T,A, lw=1.5, ls='--'):
9     t_end = T + 8*tau_max
10    t_eval = np.linspace(0.0, t_end, 4000)
11    max_step = min(T, tau_min)/20.0
12    sol = solve_ivp(f_rhs, (0.0, t_end), [0.0, 0.0], t_eval=t_eval,
13                  rtol=1e-8, atol=1e-8, max_step=max_step, args=(T,
14                        A,))
15    vin = np.zeros(len(t_eval))
16    vin[(t_eval >= 0) & (t_eval <= T)] = A
17    v2 = sol.y[1]
18    label = f"T = {T*1e3:g} ms, A = {A} V"
19    axn.plot(t_eval*1e3, v2/(A*T), lw=lw, ls=ls, label=label)
20 fig2, axn = plt.subplots(figsize=(8, 4.2))
21
22
23 simulate(100e-3, 1, lw=4, ls = "-")
24 simulate(100e-3, 5, lw = 1.5, ls = '--')
```

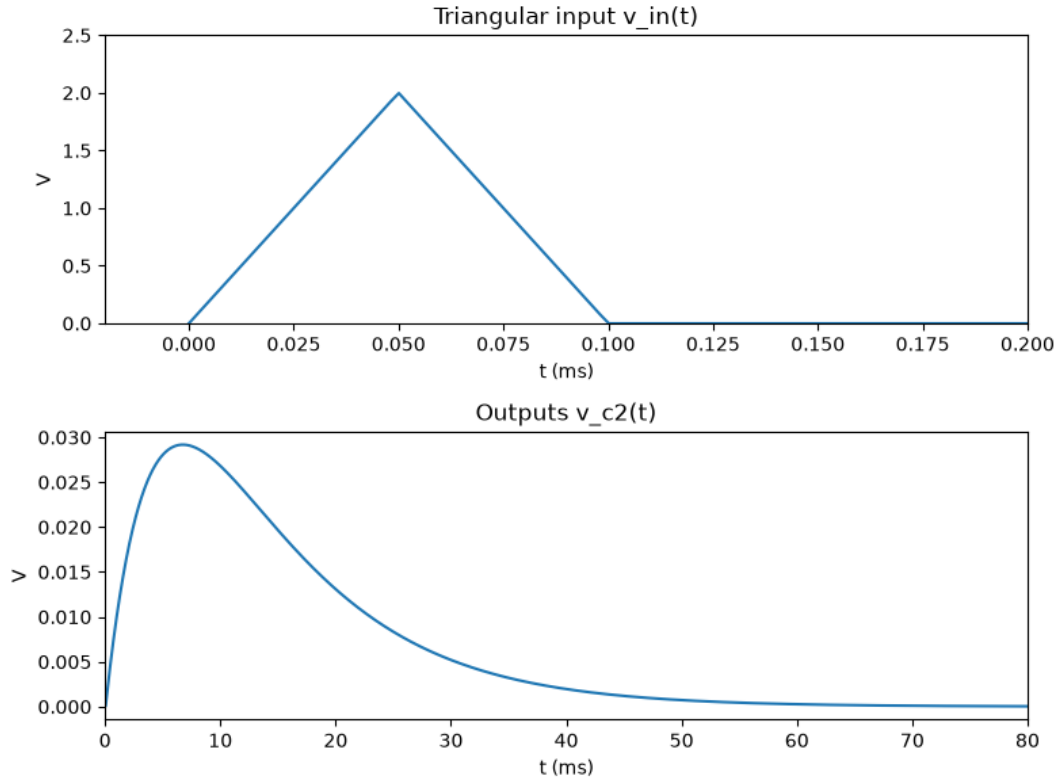
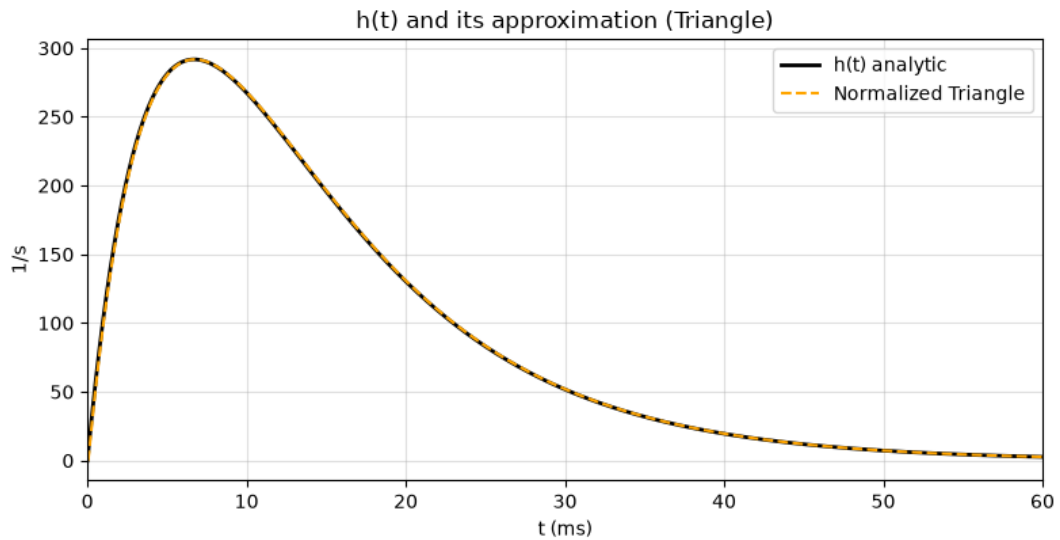
Listing 2: Modified code for comparison of area-normalized outputs for different amplitude inputs.

## Task 2.3: Triangular Pulse

For this task, the rectangular input was swapped for a triangular pulse with  $T = 100\ \mu\text{s}$  and a peak of  $A = 2\ \text{V}$ . The parts of the code that changed are below: the input array for plotting, the input inside `f_rhs` so the solver sees the triangle, and the normalization by the triangle's area.

```
1  A = 2.0          # peak of triangle (V)
2  T = 100e-6       # pulse width (s)
3
4  # triangular input array (for plotting)
5  vin = np.zeros(len(t_eval))
6  for i, t in enumerate(t_eval):
7      if 0 <= t <= T/2:
8          vin[i] = (2 * A / T) * t
9      elif T/2 < t <= T:
10         vin[i] = (2 * A / T) * (T - t)
11
12 # triangular input inside f_rhs so the solver sees it
13 def f_rhs(t, x):
14     v1, v2 = x
15     if 0 <= t <= T/2:
16         _vin = (2 * A / T) * t
17     elif T/2 < t <= T:
18         _vin = (2 * A / T) * (T - t)
19     else:
20         _vin = 0.0
21     return [(_vin - v1)/tau1, (G*v1 - v2)/tau2]
22
23 # normalize by triangle area = 0.5*A*T
24 area = 0.5 * A * T
25 axn.plot(t_eval * 1e3, v2 / area, "--", color="orange", label="
    Normalized Triangle")
```

Listing 3: Modified code for triangular input


 Figure 5: Triangular input and  $v_{c2}$  output.

 Figure 6:  $v_{c2}/(0.5AT)$  overlaid with  $h(t)$ .

The triangle has an area of  $\frac{1}{2}AT = \frac{1}{2}(2 \text{ V})(100 \mu\text{s}) = 1 \times 10^{-4} \text{ V}\cdot\text{s}$ , which is the same as the 1 V, 100  $\mu\text{s}$  rectangle from Task 2.2. Figure ?? shows the output divided by that area, and it lands right on top of  $h(t)$ . The peak is about 292 at around 6.7 ms, same as  $h(t)$ , and it stays within about 0.5% of  $h(t)$  for the rest of the curve. So no, the shape of the pulse does not matter as long as the area is the same (and the pulse is short enough).

The explanation behind this is that the output is the input convolved with  $h(t)$ :

$$v_{c2}(t) = \int_0^T h(t - \tau) v_{in}(\tau) d\tau.$$

If  $T$  is really short compared to the time constants, then  $h(t - \tau)$  barely changes while the pulse is on, so  $h(t - \tau) \approx h(t)$  for  $0 \leq \tau \leq T$ . That lets  $h(t)$  be pulled out of the integral:

$$v_{c2}(t) \approx h(t) \int_0^T v_{in}(\tau) d\tau = h(t) \cdot (\text{area of the pulse}).$$

So the only thing about the input that shows up in the output is its area, not its shape. That is why dividing by the area gives  $h(t)$  for both the rectangle ( $AT$ ) and the triangle ( $\frac{1}{2}AT$ ). The tiny difference that is left comes from the pulse not being infinitely short.

## Task 2.4: Step Response

For this task, the pulse was replaced with a step input of height  $A = 1$  V that stays on for all  $t \geq 0$ . The modified parts of the code are below: the step input, the step inside `f_rhs` so the solver sees it, and the derivative of the step response using `np.gradient()`.

```

1 A = 1.0
2 t_end = 8 * tau_max           # no pulse now, so no T added
3 t_eval = np.linspace(0.0, t_end, 4000)
4 max_step = tau_min / 20.0
5
6 # step input is A for all t >= 0
7 vin = np.ones(len(t_eval)) * A
8
9 # f_rhs with a constant step input
10 def f_rhs(t, x):
11     v1, v2 = x
12     _vin = A                # input is always A
13     return [(_vin - v1)/tau1, (G*v1 - v2)/tau2]
14
15 sol = solve_ivp(f_rhs, (0.0, t_end), [0.0, 0.0], t_eval=t_eval, rtol
16               =1e-8, atol=1e-8, max_step=max_step)
17 v2_step = sol.y[1]
18
19 # differentiate the step response
20 ds_dt = np.gradient(v2_step, t_eval)

```

Listing 4: Modified code for step input and differentiation

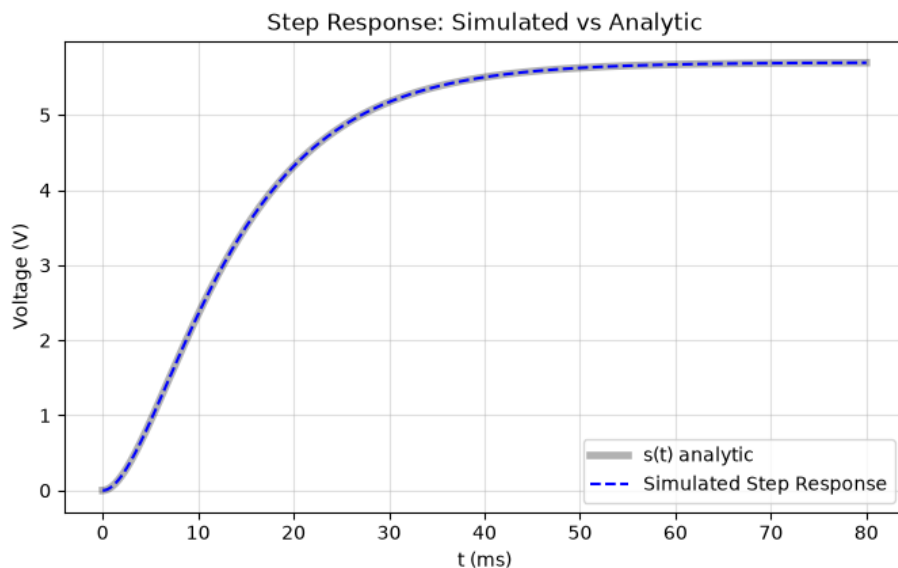


Figure 7: Simulated step response with analytic  $s(t)$  overlaid.

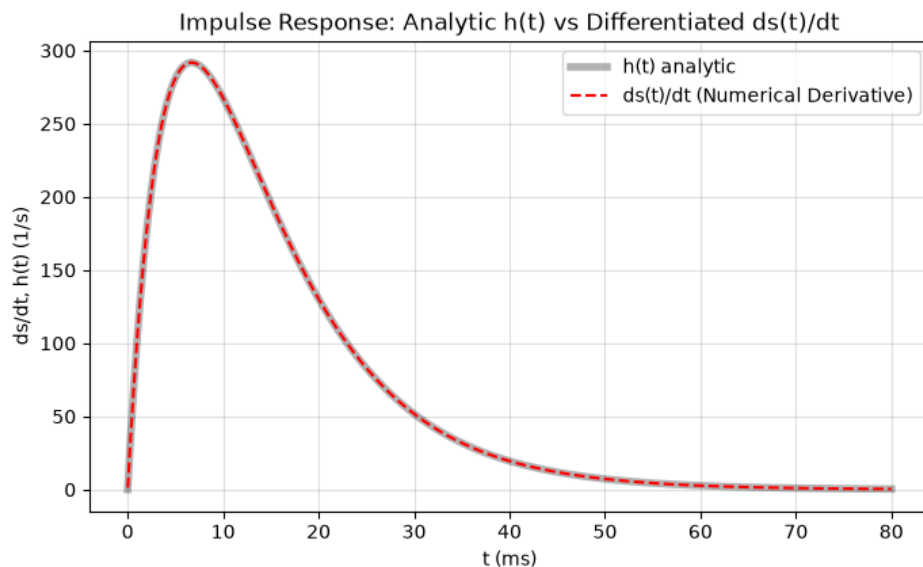


Figure 8:  $h(t)$  overlaid with  $ds(t)/dt$  computed numerically.

Figure ?? shows the simulated step response with the analytic  $s(t)$  from Task 1 behind it. The two curves sit right on top of each other, so the simulation matches the math.

### Are $h(t)$ and $ds(t)/dt$ the same?

Yes. In Figure ??, the numerical derivative of the step response lands right on top of the analytic  $h(t)$ . This makes sense because the step response is the integral of the impulse response,

$$s(t) = \int_0^t h(\sigma) d\sigma,$$

so taking the derivative of  $s(t)$  just gives  $h(t)$  back:

$$\frac{ds(t)}{dt} = h(t).$$

### Final value and settling time

The step response levels off at about 5.696 V by  $t = 80$  ms, which is  $G = 5.7$  to within 0.1%. The tiny gap is just the last bit of the slow exponential not being gone yet. This matches the DC gain from Task 1,  $G = 1 + R_5/R_4 = 5.7$ , so a 1 V step ends up as 5.7 V at the output.

The settling time is mostly controlled by the slower time constant,  $\tau_1 = 10$  ms. Writing  $s(t)$  as

$$s(t) = G - \frac{G}{\tau_1 - \tau_2} (\tau_1 e^{-t/\tau_1} - \tau_2 e^{-t/\tau_2}),$$

the  $e^{-t/\tau_2}$  term dies off faster, so at later times the gap to  $G$  is mostly just the  $e^{-t/\tau_1}$  term. From the simulation, the output gets within 2% of  $G$  at about 45 ms and within 1% at about 52 ms. For a single RC circuit, 2% settling would be about  $4\tau_1 = 40$  ms, so the second stage makes it take a little longer. The faster time constant  $\tau_2$  does not really set the settling time. It mostly shows up at the start, where the response begins flat and then curves upward instead of jumping up like a single RC circuit.

### Task 3.1: Build and Instrument

The RC opamp circuit was built before any measurements were taken and everything was looked over by the group, the first build of the circuit was not working properly, due to a swapped resistor making the gain not boost to 5.7. Once that was resolved the opamp was also replaced just to be safe and the output was amplified as expected.

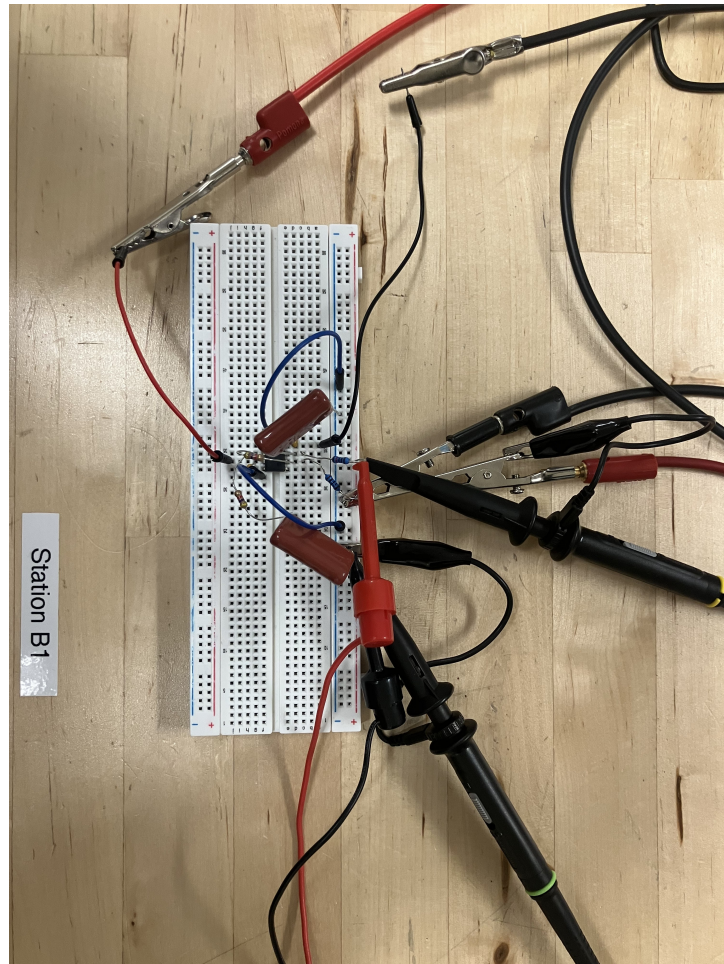


Figure 9: RC Opamp Circuit on Breadboard.

## Task 3.2: Impulse Response Measurements

5 V, 100  $\mu$ s, 10 Hz

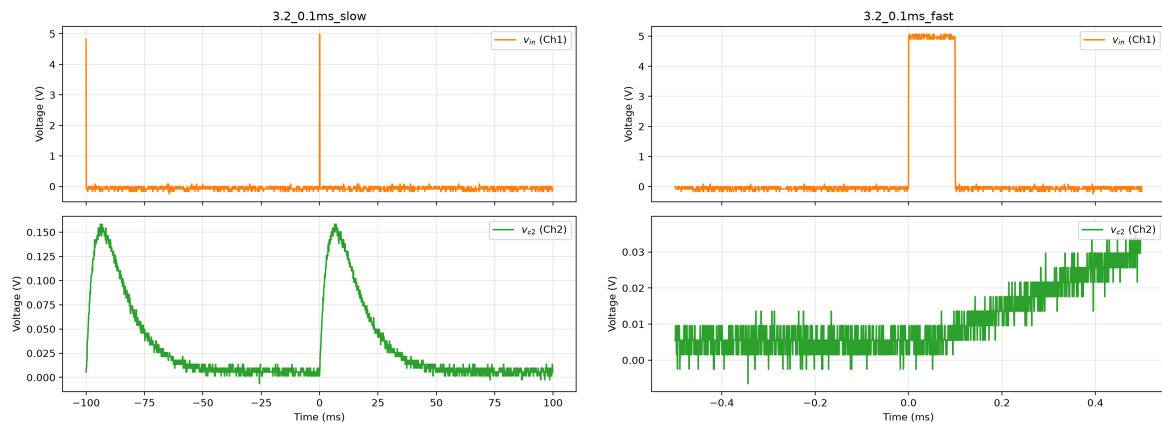


Figure 10: Slow and fast sweeps for the 100  $\mu$ s pulse.

5 V, 1 ms, 10 Hz

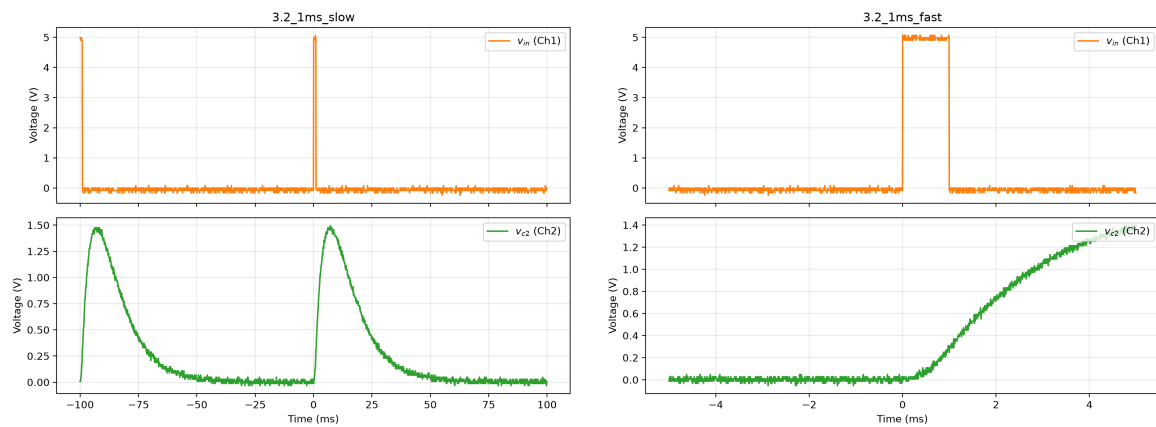


Figure 11: Slow and fast sweeps for the 1 ms pulse.

## 2 V, 10 ms, 10 Hz

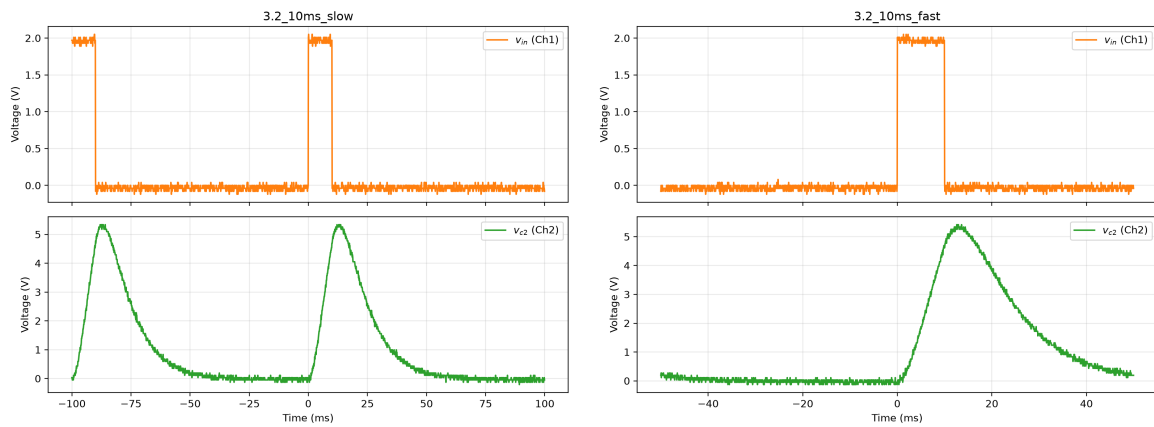


Figure 12: Slow and fast sweeps for the 10 ms pulse.

## 2 V, 50 ms, 1 Hz

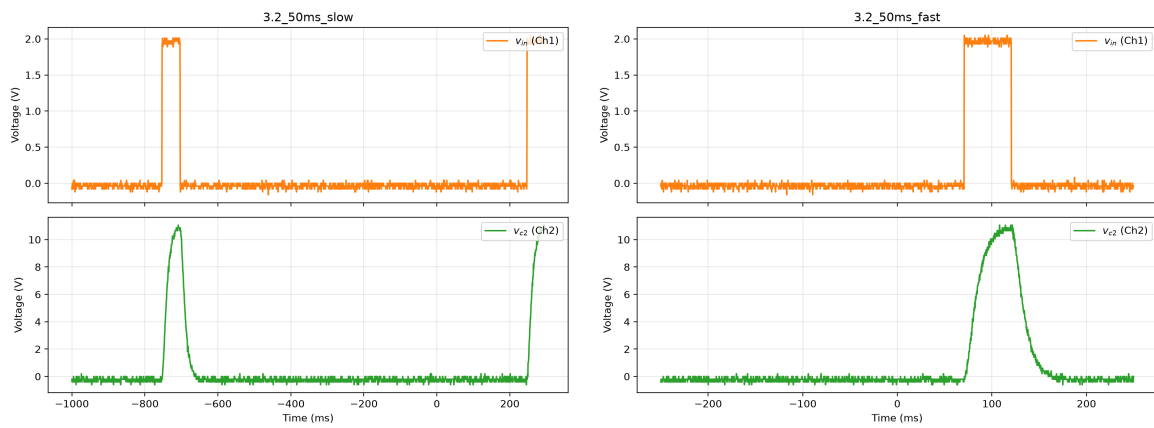


Figure 13: Slow and fast sweeps for the 50 ms pulse.

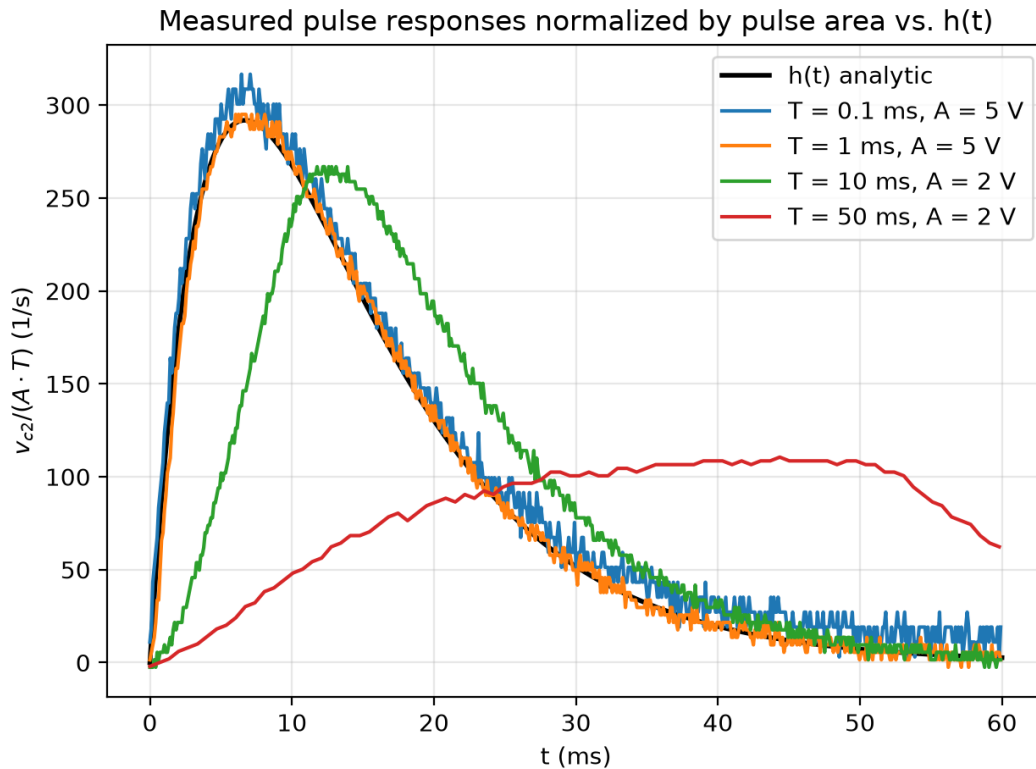
Normalized traces vs.  $h(t)$ 

Figure 14: Four measured  $v_{c2}(t)$  traces normalized by pulse area, with analytic  $h(t)$ .

**Agreement with  $h(t)$ :** The  $100 \mu\text{s}$  and the  $1 \text{ ms}$  pulses have the best agreement with  $h(t)$ , the peaks and timing of the other graphs don't line up nearly as well, as the pulse no longer approximates an impulse at larger widths. FINISH is due to the signal not acting like an impulse at the higher widths, so the circuit has time to respond. Errors on top of this point to component tolerances, oscilloscope measurements, and a non-ideal pulse from the function generator.

### Task 3.3: Step Response Measurements

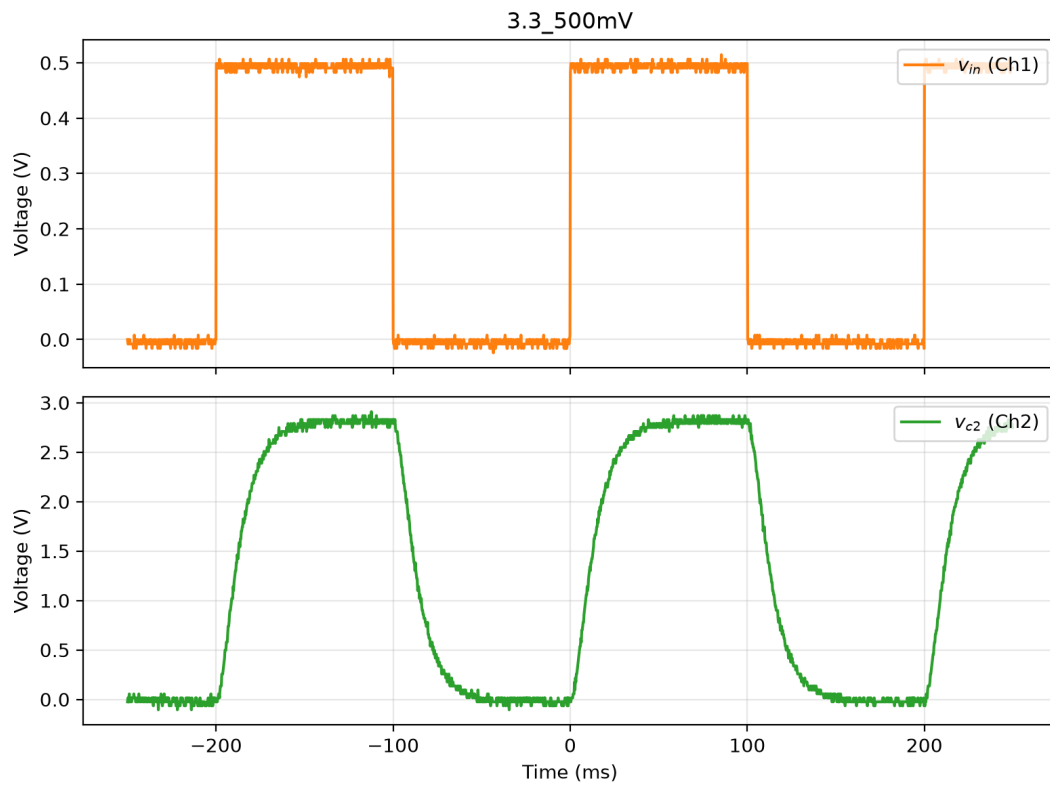


Figure 15: Measured response to a 500 mV step (same rise and fall time constants).

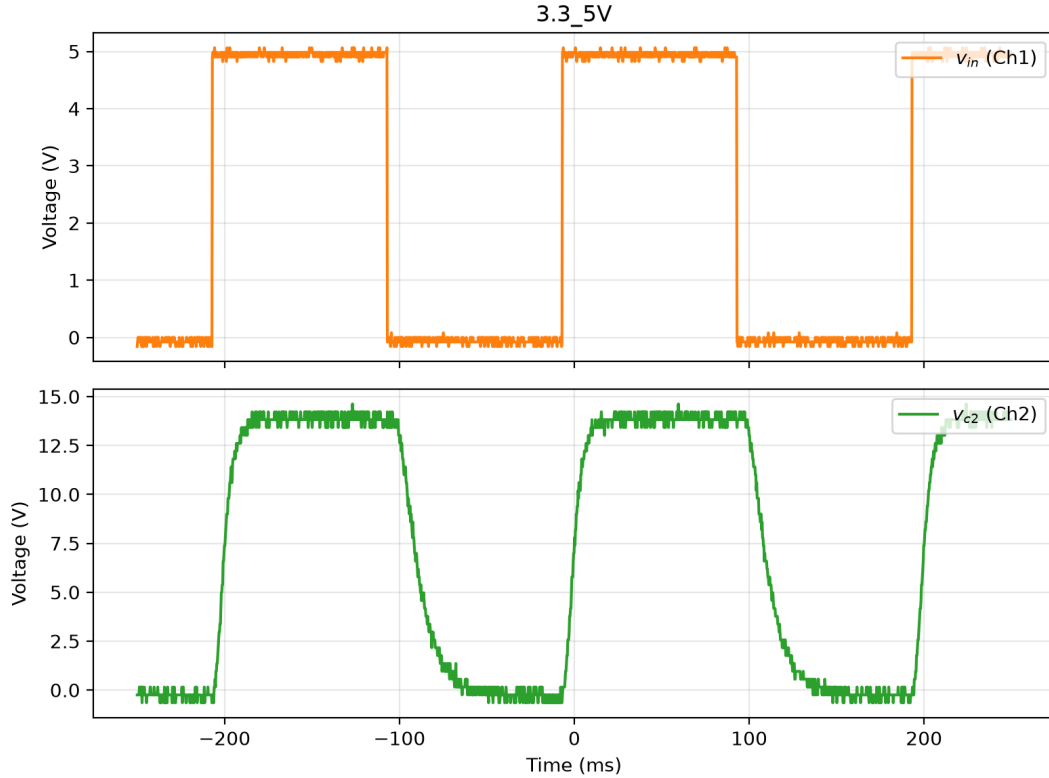


Figure 16: Measured response to a 5 V step (asymmetric rising/falling edges).

The 500 mV step produced a smooth response with matching rise and fall behavior (Figure ??). The output settled at about 2.82 V, giving a measured gain of about 5.6, close to the theoretical  $G = 5.7$ . Because the output stays well within the op-amp's output range, the circuit behaves linearly and the rising and falling edges are mirror-image exponentials.

For the 5 V step (Figure ??), the linear prediction for the output is  $G \times 5 \text{ V} = 28.5 \text{ V}$ , but the output instead flattens at about 13.9 V, an apparent gain of only about 2.8. This is op-amp output saturation. When LM741 is powered from  $\pm 15 \text{ V}$ , the output typically swings at approximately  $\pm 14 \text{ V}$ . Once the output reaches this limit, the negative feedback can no longer hold the inverting input equal to the non-inverting input, so the relation  $v_{buf} = G v_{c1}$  breaks down and the op-amp output stays pinned at the rail. Slew rate is not the cause here, since the LM741 slews at about  $0.5 \text{ V } \mu\text{s}^{-1}$ , far faster than the millisecond-scale changes in this circuit.

Saturation begins when  $G v_{c1}$  exceeds the rail, i.e. when

$$v_{c1} > \frac{13.9 \text{ V}}{5.7} \approx 2.44 \text{ V}.$$

For the 500 mV step,  $v_{c1}$  never exceeds 0.5 V, so the op-amp stays linear. For the 5 V step,  $v_{c1}$  rises well past this threshold, which causes the asymmetry between the edges. On the rising edge, once  $v_{c1}$  crosses this threshold,  $v_{buf}$  is fixed at 13.9 V. The second RC stage then simply charges toward this constant value, so the top of the  $v_{c2}$  rise flattens out abruptly rather than curving smoothly toward its final value.

On the falling edge,  $v_{in}$  drops to zero immediately, but  $v_{c1}$  decays from about 5 V through the first RC stage. Until  $v_{c1}$  falls back below 2.44 V,  $G v_{c1}$  is still above the rail and the op-amp output remains momentarily saturated, so there is a delay before

$v_{c2}$  begins to fall. This delay is visible in Figure ??, where  $v_{c2}$  begins falling several milliseconds after  $v_{in}$  drops, while for the 500 mV step it begins falling immediately. The asymmetry is therefore a result of  $v_{c1}$  overshooting the saturation threshold: on the way up the output is clipped partway through the rise, and on the way down the excess voltage on  $C_1$  must discharge before the output can respond.

### Task 3.4: Frequency Response Measurements

Table 1: Measured amplitude and time shift of  $v_{c1}$  and  $v_{c2}$ .

| $f$ (Hz) | $V_{c1}$ (pp) | $\Delta t_1$ (ms) | $V_{c2}$ (pp) | $\Delta t_2$ (ms) |
|----------|---------------|-------------------|---------------|-------------------|
| 5        | 0.94          | 9.5               | 5.37          | 14.2              |
| 15       | 0.71          | 7.9               | 3.80          | 12.4              |
| 30       | 0.47          | 5.8               | 2.01          | 9.5               |
| 60       | 0.25          | 3.5               | 0.71          | 6.3               |
| 120      | 0.13          | 1.9               | 0.20          | 3.6               |

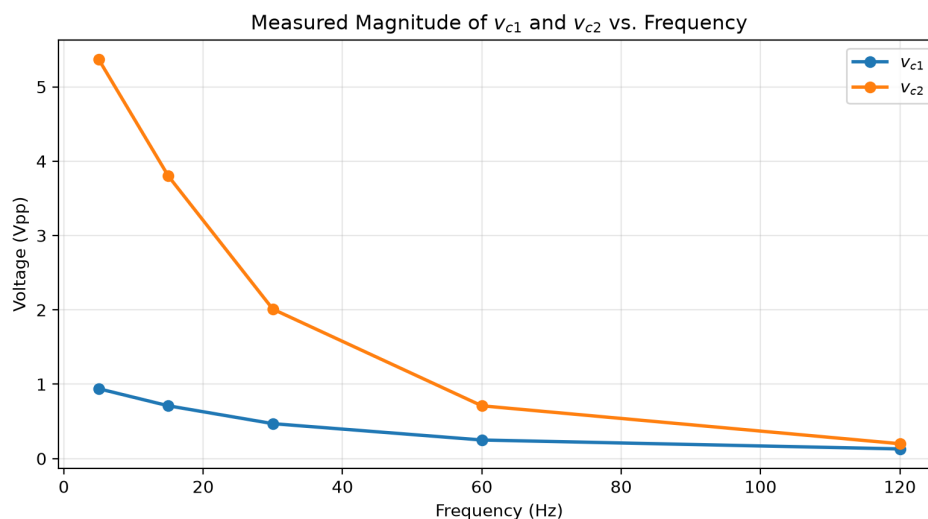


Figure 17: Measured magnitude of  $v_{c1}$  and  $v_{c2}$  versus frequency.

$V_{c2}$  has a much faster roll-off. Starting at 5 Hz,  $V_{c2}$  is at approximately 5.4 V, while  $V_{c1}$  is at approximately 1 V. However, by 60 Hz, the amplitude of  $V_{c2}$  decreased by nearly 4 V to under 1 V, whereas  $V_{c1}$  only decreased by a little over 0.5 V. By 120 Hz, the magnitude of  $V_{c2}$  is only 0.07 V greater than  $V_{c1}$ , despite starting over 4 V larger in magnitude at 5 Hz.

## Conclusion

In our lab, we analyzed the two-stage RC cascade filter described by Figure ?? by hand. We found the ODE which describes the two-stage RC circuit to be:

$$\ddot{v}_{c2}(t) + \left(\frac{1}{\tau_1} + \frac{1}{\tau_2}\right) \dot{v}_{c2}(t) + \frac{1}{\tau_1 \tau_2} v_{c2}(t) = \frac{G}{\tau_1 \tau_2} v_{in}(t).$$

We determined the impulse response for  $v_{c2}(t)$  to be

$$h(t) = \frac{G}{\tau_1 - \tau_2} (e^{-t/\tau_1} - e^{-t/\tau_2}) u(t).$$

We calculated the step response for the two-stage RC circuit to be:

$$s(t) = \frac{G}{\tau_1 - \tau_2} \left[ \tau_1 (1 - e^{-t/\tau_1}) + \tau_2 (e^{-t/\tau_2} - 1) \right]$$

. We computed the following DC gain for the two-stage RC circuit:

$$G = 1 + \frac{R_5}{R_4}.$$

Using  $s(t)$ , when  $t \geq T$  we found  $v_{out}(t)$  to be

$$v_{out}(t) = \frac{AG}{\tau_1 - \tau_2} \left[ \tau_1 (e^{-(t-T)/\tau_1} - e^{-t/\tau_1}) + \tau_2 (e^{-t/\tau_2} - e^{-(t-T)/\tau_2}) \right].$$

Finally, we calculated that  $T$  can be at most  $94 \mu\text{s}$  for  $v_{out}(t)$  to approximate  $h(t)$  within a 1% error.

After performing hand analysis, we conducted python simulations of output of the two-stage RC cascade filter for input pulses of amplitude  $A = 5 \text{ V}$  with different lengths and compared these to the impulse response  $h(t)$ . We found that a short pulse of  $0.1 \text{ ms}$  collapses almost perfectly onto  $h(t)$ , a slightly longer pulse of  $1 \text{ ms}$  has a 5–10% error compared to  $h(t)$ , and larger pulses of  $10 \text{ ms}$  and  $50 \text{ ms}$  don't collapse onto  $h(t)$  whatsoever. These findings confirmed our calculation that pulse width  $T$  cannot be greater than  $94 \mu\text{s}$  to achieve an approximation of  $h(t)$  with a 1% error or less. Next, we graphed the response of the two-stage RC filter when an input pulse of  $A = 1 \text{ V}$ ,  $T = 100 \text{ ms}$  and compared it to the response of the circuit with an input pulse of  $A = 5 \text{ V}$ ,  $T = 100 \text{ ms}$ . The  $5 \text{ V}$  output was exactly 5 times larger than the  $1 \text{ V}$  output which demonstrated the linearity of the circuit. Then, we modified our code to give the simulated filter a  $T = 100 \mu\text{s}$ ,  $A = 2 \text{ V}$  triangle pulse input. We found that the resulting graph matched exactly with a square pulse of the same area which demonstrated that as  $T \rightarrow 0$  the response of the filter depends solely on the area, not the shape of the pulse. Finally, we modified the code to apply a step input to the simulated filter instead of a pulse input, and found the step response  $s(t)$ . We then differentiated the simulated step response and found that it exactly matched the graph of our analytical impulse response  $h(t)$ . We confirmed that the final step response equals op-amp gain  $G$  to within 0.1% and determined that the settling time depends mainly on  $\tau_1$  opposed to  $\tau_2$ .

When we completed our numerical simulations we built the two-stage RC cascade filter on a breadboard. We graphed the responses of the circuit for the following pulse inputs: [amplitude =  $5 \text{ V}$ , offset =  $2.5 \text{ V}$ , width =  $100 \mu\text{s}$ ,  $f = 10 \text{ Hz}$ ], [amplitude =  $5 \text{ V}$ , offset =

2.5 V, width = 1 ms,  $f = 10$  Hz], [amplitude = 2 V, offset = 1 V, width = 10 ms,  $f = 10$  Hz], and [amplitude = 2 V, offset = 1 V, width = 50 ms,  $f = 1$  Hz]. We found, similarly to our numerical simulations, that a pulse of width close to  $T = 94\text{ }\mu\text{s}$  yields an output that closely matches  $h(t)$  while pulses of higher widths result in outputs that start to greatly drift away from  $h(t)$ , with the 50 ms pulse even approximating the step response  $s(t)$  instead. After, we gave the circuit pulses of amplitude = 500 mV, offset = 250 mV, width = 100 ms,  $f = 5$  Hz to analyze the step response of the circuit and observed that the rising and falling edges of the step response were mirrored. We then increased the amplitude of the pulses, while keeping the width and frequency constant, to amplitude = 5 V, offset = 2.5 V, width = 100 ms,  $f = 5$  Hz and found that the rising and falling edges of our output were no longer mirrored, due to oversaturation of the op-amp. Finally, we drove the circuit with sinusoidal inputs with amplitude 1 V peak-to-peak and 0 offset. We recorded the peak-to-peak amplitude and time shift of both  $v_{c1}$  and  $v_{c2}$  for frequencies of 5, 15, 30, 60, 120 Hz. We found that  $v_{c2}$  has a much higher roll-off than  $v_{c1}$ .

We had several key takeaways about impulse responses, step responses, and linear systems from this lab. First, we found that for very small widths, a pulse input can allow us to approximate the impulse response of a system. Second, we found that the first statement holds true regardless of input shape as  $T \rightarrow 0$ . Third, we found from this lab that complex filter circuits including multiple resistors, capacitors, and an op-amp such as the two-stage RC cascade we analyzed can be modeled as LTI systems (at least given inputs that don't oversaturate the op-amp). Fourth, we confirmed for our simulated circuits that the impulse response is the derivative of the step response. Finally, in a cascaded RC filter the settling time is determined primarily by the slower time constant in the cascade.

Overall, our experimental results aligned closely with our analytical and numerical results. Minor errors may be a result of non-ideal components, stray capacitance and inductance of the breadboard and wires.

Our group feels that our understanding of step responses, impulse responses, and circuits modeled by LTI systems was greatly enhanced by this lab experience.

# Appendix

## Task 3 Code

```

1 import glob, os
2 import pandas as pd
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6
7 FOLDER = os.path.dirname(os.path.abspath(__file__)) # put this
               script in the CSV folder
8 OUT = os.path.join(FOLDER, "graphs") # separate folder so scope .
               png files never get overwritten
9 os.makedirs(OUT, exist_ok=True)
10
11 for path in sorted(glob.glob(os.path.join(FOLDER, "*.csv"))):
12     name = os.path.splitext(os.path.basename(path))[0]
13
14     # skip the scope header, data starts at the Sample Number line
15     with open(path, encoding="utf-8-sig") as f:
16         skip = next(i for i, line in enumerate(f) if line.startswith(
17             "Sample Number"))
18     df = pd.read_csv(path, skiprows=skip, encoding="utf-8-sig").
19         dropna(axis=1, how="all")
20
21     t = df["Time (s)"] * 1e3 # ms
22     ch2_label = r"$v_{c1}$" if "vc1" in name else r"$v_{c2}$"
23
24     fig, ax = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
25
26     ax[0].plot(t, df.iloc[:, 2], color="tab:orange", label=r"$v_{in}$
27         (Ch1)")
28     ax[0].set_ylabel("Voltage (V)")
29     ax[0].set_title(name)
30     ax[0].grid(True, alpha=0.3)
31     ax[0].legend(loc="upper right")
32
33     ax[1].plot(t, df.iloc[:, 3], color="tab:green", label=f"{
34         ch2_label} (Ch2)")
35     ax[1].set_xlabel("Time (ms)")
36     ax[1].set_ylabel("Voltage (V)")
37     ax[1].grid(True, alpha=0.3)
38     ax[1].legend(loc="upper right")
39
40     fig.tight_layout()
41     fig.savefig(os.path.join(OUT, name + ".png"), dpi=200)
42     plt.close(fig)
43     print("saved", name)
44
45 # Task 3.4 frequency response plot
46 frequency = [5, 15, 30, 60, 120]
47 vc1 = [0.94, 0.71, 0.47, 0.25, 0.13]
48 vc2 = [5.37, 3.80, 2.01, 0.71, 0.20]
49
50 fig, ax = plt.subplots(figsize=(8, 4.5))

```

```
48
49 ax.plot(frequency, vc1, marker="o", linewidth=2, label=r"$v_{c1}$")
50 ax.plot(frequency, vc2, marker="o", linewidth=2, label=r"$v_{c2}$")
51
52 ax.set_xlabel("Frequency (Hz)")
53 ax.set_ylabel("Voltage (Vpp)")
54 ax.set_title("Measured Magnitude of $v_{c1}$ and $v_{c2}$ vs.
    Frequency")
55 ax.grid(True, alpha=0.3)
56 ax.legend()
57
58 fig.tight_layout()
59 fig.savefig(os.path.join(OUT, "task3_4_frequency_response.png"), dpi
    =200)
60 plt.close(fig)
```

Listing 5: Python code used for graphing raw CSVs for Part 3, as well as table data for Task 3.4.

```
1 import numpy as np, pandas as pd, matplotlib.pyplot as plt
2
3 tau1, tau2, G = 10e-3, 4.7e-3, 5.7
4 t = np.linspace(0, 0.06, 600)
5 h = G/(tau1-tau2) * (np.exp(-t/tau1) - np.exp(-t/tau2))
6 plt.plot(t*1e3, h, "k", lw=2, label="h(t) analytic")
7
8 # (file, amplitude, width, legend label)
9 runs = [("3.2_0.1ms_slow.csv", 5, 100e-6, "T = 0.1 ms, A = 5 V"),
10         ("3.2_1ms_slow.csv", 5, 1e-3, "T = 1 ms, A = 5 V"),
11         ("3.2_10ms_slow.csv", 2, 10e-3, "T = 10 ms, A = 2 V"),
12         ("3.2_50ms_slow.csv", 2, 50e-3, "T = 50 ms, A = 2 V")]
13
14 for f, A, T, label in runs:
15     df = pd.read_csv(f, skiprows=7, encoding="utf-8-sig") # data
16     # starts after the header
17     time, vin, vout = df.iloc[:, 1], df.iloc[:, 2], df.iloc[:, 3]
18     t0 = time[vin > A/2].iloc[0] # rising
19     # edge of pulse
20     m = (time >= t0) & (time <= t0 + 0.06)
21     plt.plot((time[m] - t0)*1e3, vout[m]/(A*T), label=label)
22
23 plt.title("Measured pulse responses normalized by pulse area vs. h(t)
24 ")
25 plt.xlabel("t (ms)")
26 plt.ylabel(r"$v_{c2}/(A \cdot T)$ (1/s)")
27 plt.legend()
28 plt.grid(alpha=.3)
29 plt.tight_layout()
30 plt.savefig("3.2_normalized.png", dpi=200)
```

Listing 6: Python code for graphing normalized graph for Task 3.2.